SQL - Funkcje okna (Window functions)
Lab 2

Imiona i nazwiska: Karolina Węgrzyn, Patrycja Markiewicz

Celem ćwiczenia jest zapoznanie się z działaniem funkcji okna (window functions) w SQL, analiza wydajności zapytań i porównanie z rozwiązaniami przy wykorzystaniu "tradycyjnych" konstrukcji SQL

Swoje odpowiedzi wpisuj w miejsca oznaczone jako:

> Wyniki:

```
-- ...
```

Ważne/wymagane są komentarze.

Zamieść kod rozwiązania oraz zrzuty ekranu pokazujące wyniki, (dołącz kod rozwiązania w formie tekstowej/źródłowej)

Zwróć uwagę na formatowanie kodu

Oprogramowanie - co jest potrzebne?

Do wykonania ćwiczenia potrzebne jest następujące oprogramowanie:

- MS SQL Server - wersja 2019, 2022
- PostgreSQL - wersja 15/16/17
- SQLite
- Narzędzia do komunikacji z bazą danych
    - SSMS - Microsoft SQL Managment Studio
    - DtataGrip lub DBeaver
- Przykładowa baza Northwind/Northwind3
    - W wersji dla każdego z wymienionych serwerów

Oprogramowanie dostępne jest na przygotowanej maszynie wirtualnej

Dokumentacja/Literatura

- Kathi Kellenberger, Clayton Groom, Ed Pollack, Expert T-SQL Window Functions in SQL Server 2019, Apres 2019

- Itzik Ben-Gan, T-SQL Window Functions: For Data Analysis and Beyond, Microsoft 2020

- Kilka linków do materiałów które mogą być pomocne - https://learn.microsoft.com/en-us/sql/t-sql/queries/select-over-clause-transact-sql?view=sql-server-ver16
    - https://www.sqlservertutorial.net/sql-server-window-functions/
    - https://www.sqlshack.com/use-window-functions-sql-server/
    - https://www.postgresql.org/docs/current/tutorial-window.html
    - https://www.postgresqltutorial.com/postgresql-window-function/
    - https://www.sqlite.org/windowfunctions.html
    - https://www.sqlitetutorial.net/sqlite-window-functions/
- W razie potrzeby - opis Ikonek używanych w graficznej prezentacji planu zapytania w SSMS jest tutaj:
    - https://docs.microsoft.com/en-us/sql/relational-databases/showplan-logical-and-physical-operators-reference

Przygotowanie

Uruchom SSMS - Skonfiguruj połączenie z bazą Northwind na lokalnym serwerze MS SQL

Uruchom DataGrip (lub Dbeaver)

- Skonfiguruj połączenia z bazą Northwind3
    - na lokalnym serwerze MS SQL
    - na lokalnym serwerze PostgreSQL
    - z lokalną bazą SQLite

Można też skorzystać z innych narzędzi klienckich (wg własnego uznania)

Oryginalna baza Northwind jest bardzo mała. Warto zaobserwować działanie na nieco większym zbiorze danych.

Korzystamy ze "zmodyfikowanej wersji" bazy northwind

Baza Northwind3 zawiera dodatkową tabelę product_history

- 2,2 mln wierszy

Bazę Northwind3 można pobrać z moodle (zakładka - Backupy baz danych)

Zadanie 1

Funkcje rankingu, `row_number()`, `rank()`, `dense_rank()`

```
select productid, productname, unitprice, categoryid,
    row_number() over(partition by categoryid order by unitprice desc) as
rowno,
    rank() over(partition by categoryid order by unitprice desc) as
rankprice,
    dense_rank() over(partition by categoryid order by unitprice desc) as
```

```
denserankprice
from products;
```

Wykonaj polecenie, zaobserwuj wynik. Porównaj funkcje row_number(), rank(), dense_rank(). Skomentuj wyniki.

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

> Wyniki:

MS SQL Server

```
select productid, productname, unitprice, categoryid,
    row_number() over(partition by categoryid order by unitprice desc) as
rowno,
    rank() over(partition by categoryid order by unitprice desc) as
rankprice,
    dense_rank() over(partition by categoryid order by unitprice desc) as
denserankprice
from products;
```

	productid	productname	unitprice	categoryid	rowno	rankprice	denserankprice
1	38	Côte de Blaye	263.5000	1	1	1	1
2	43	Ipoh Coffee	46.0000	1	2	2	2
3	2	Chang	19.0000	1	3	3	3
4	1	Chai	18.0000	1	4	4	4
5	39	Chartreuse verte	18.0000	1	5	4	4
6	35	Steeleye Stout	18.0000	1	6	4	4
7	76	Lakkalikööri	18.0000	1	7	4	4
8	70	Outback Lager	15.0000	1	8	8	5
9	67	Laughing Lumberjack Lager	14.0000	1	9	9	6
10	34	Sasquatch Ale	14.0000	1	10	9	6
11	75	Rhönbräu Klosterbier	7.7500	1	11	11	7
12	24	Guaraná Fantástica	4.5000	1	12	12	8
13	63	Vegie-spread	43.9000	2	1	1	1
14	8	Northwoods Cranberry Sauce	40.0000	2	2	2	2
15	61	Sirop d'érable	28.5000	2	3	3	3

Tabela jest dzielona na okna dla każdej kategorii, a wewnątrz każdej grupy produkty zostaja posortowane malejąco po cenie.

Funkcja `row_number()` zwraca kolejne, unikalne liczby całkowite, ignorując remisy, dlatego wynikiem działania tej funkcji są cyfry od 1 do liczby unikalnych produktó w kategorii.

Funkcja `rank()` nadaje kolejne pozycje, ale uwzględniając remisy. Na powyższym screenie możemy zauważyć, że 4 produkty mają przyporządkowane miejsce 4. Co istotne, następny element po remisie w przypadku tej funkcji nie jest kolejną liczbą porządkową, tylko liczbą, która pomija tyle miejsc, ile wierszy współdzieliło poprzednią pozycję - dlatego w tym konkretnym przypadku jest to wartość 8.

Funkcja `dense_rank()` podobnie jak `rank()`, uwzględnia remisy, ale nie tworzy luk w numeracji. Czyli po czterech produktach remisujących na miejscu 4, następny w kolejności produkt otrzymuje miejsce 5, czyli kolejną liczbę porządkowa.

Polecenie równoważne bez użycia funkcji okna.

```sql
select
    productid,
    productname,
    unitprice,
    categoryid,

    (select count(*) + 1
     from products p2
     where p2.categoryid = p1.categoryid
       and (p2.unitprice > p1.unitprice
                or (p2.unitprice = p1.unitprice
                        and p2.productid < p1.productid))
    ) as rowno,

    (select count(*) + 1
    from products p2
    where p2.categoryid = p1.categoryid
      and p2.unitprice > p1.unitprice
    ) as rankprice,

    (select count(distinct p2.unitprice) + 1
     from products p2
     where p2.categoryid = p1.categoryid
       and p2.unitprice > p1.unitprice
     ) as denserankprice

from products p1
order by p1.categoryid, rowno
```

	productid	productname	unitprice	categoryid	rowno	rankprice	denserankprice
1	38	Côte de Blaye	263.5000	1	1	1	1
2	43	Ipoh Coffee	46.0000	1	2	2	2
3	2	Chang	19.0000	1	3	3	3
4	1	Chai	18.0000	1	4	4	4
5	35	Steeleye Stout	18.0000	1	5	4	4
6	39	Chartreuse verte	18.0000	1	6	4	4
7	76	Lakkalikööri	18.0000	1	7	4	4
8	70	Outback Lager	15.0000	1	8	8	5
9	34	Sasquatch Ale	14.0000	1	9	9	6
10	67	Laughing Lumberjack Lager	14.0000	1	10	9	6
11	75	Rhönbräu Klosterbier	7.7500	1	11	11	7
12	24	Guaraná Fantástica	4.5000	1	12	12	8
13	63	Vegie-spread	43.9000	2	1	1	1

Jak widać wynik jest taki sam jak w przypadku funkcji okna.

Metoda	Koszt	Czas (ms)
Podzapytania	0.964	2.0
Funkcje okna	0.016	0.0

Wersja z podzapytaniami jest wielokrotnie droższa dla silnika bazy danych ponieważ przy podzapytaniach baza musi przeliczać całą tabelę dla każdego pojedynczego wiersza. Funkcje okna wypadają dużo lepiej.

Zadanie 2

Baza: Northwind, tabela product_history

Dla każdego produktu, podaj 4 najwyższe ceny tego produktu w danym roku. Zbiór wynikowy powinien zawierać:

- rok
- id produktu
- nazwę produktu
- cenę
- datę (datę uzyskania przez produkt takiej ceny)
- pozycję w rankingu
- Uporządkuj wynik wg roku, nr produktu, pozycji w rankingu

W przypadku długiego czasu wykonania ogranicz zbiór wynikowy.

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna, porównaj wyniki, czasy i plany zapytań (koszty).

Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSql, SQLite)

> Wyniki:

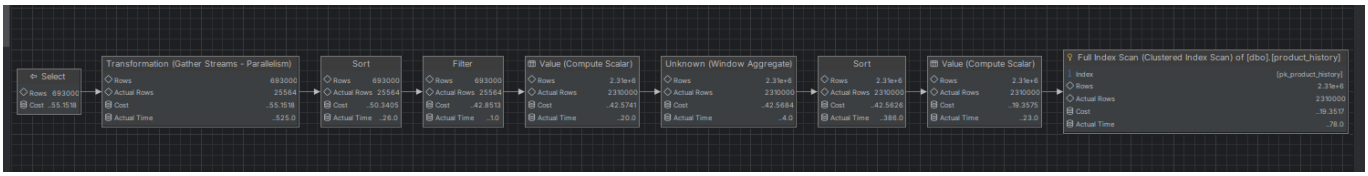
MS SQL Server

```
;WITH rankedprices AS (
    SELECT
        YEAR(date) as year,
        productid,
        productname,
        unitprice,
        date,
        ROW_NUMBER() OVER (
            PARTITION BY productid, YEAR(date)
            ORDER BY unitprice DESC
            ) AS ranknumber
    FROM product_history
)
SELECT * FROM rankedprices
WHERE ranknumber <= 4
ORDER BY year, productid, ranknumber;
```

Session | north.dbo.product_history | Result 38

Output | Plan

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
Select		693000		55.1518		BatchModeOnRowS...
Transformation (Gather Streams - Parall		693000	25564	55.1518	525.0	AvgRowSize = 59;E...
Sort		693000	25564	50.3405	26.0	AvgRowSize = 59;E...
Filter		693000	25564	42.8513	1.0	AvgRowSize = 59;E...
Value (Compute Scalar)		2.31E+6	2310000	42.5741	20.0	AvgRowSize = 59;E...
Unknown (Window Aggrega		2.31E+6	2310000	42.5684	4.0	AvgRowSize = 55;E...
Sort		2.31E+6	2310000	42.5626	386.0	AvgRowSize = 51;E...
Value (Compute S		2.31E+6	2310000	19.3575	23.0	AvgRowSize = 51;E...
Full Index Scan	table: [dbo].[product_history]; index: [pk_product_history];	2.31E+6	2310000	19.3517	78.0	AvgRowSize = 47;E...




```sql
SELECT
    ph1.year,
    ph1.productid,
    ph1.productname,
    ph1.unitprice,
    ph1.date,
    (
        SELECT COUNT(*)
        FROM product_history ph2
        WHERE ph2.productid = ph1.productid
          AND YEAR(ph2.date) = ph1.year
          AND ph2.unitprice >= ph1.unitprice
    ) AS ranknumber
FROM (
          SELECT
              productid,
              YEAR(date) AS year,
              unitprice,
              productname,
              date
          FROM product_history
     ) ph1
WHERE (
          SELECT COUNT(*)
          FROM product_history ph2
          WHERE ph2.productid = ph1.productid
            AND YEAR(ph2.date) = ph1.year
            AND ph2.unitprice > ph1.unitprice
      ) < 4
ORDER BY ph1.year, ph1.productid, ranknumber;
```

Dla całego zbioru danych udało się tylko uzyskać wyniki dla funkcji okna:

- Total Cost: 55.1518
- Actual Total Time: 525.0

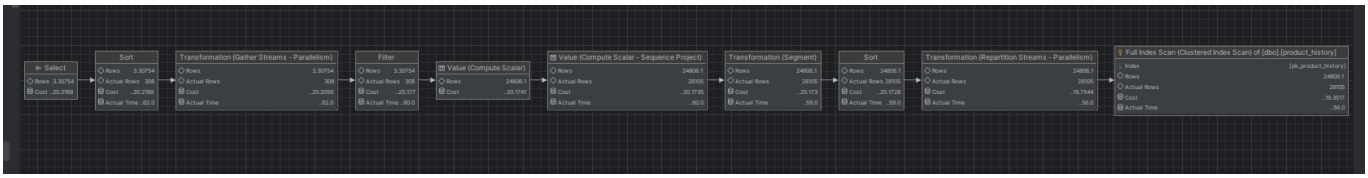
Drugi sposób jest bardzo nieefektywny - nie udało się uzyskać wyniku zapytania. Wyniki pomiędzy dwoma sposobami porównamy tylko dla jednego wybranego roku, tj. 1997.

Session | north.dbo.product_history | Result 4

Output | Plan

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
Select		3.30754		20.2169		CardinalityEstimation...
Sort		3.30754	308	20.2169	62.0	AvgRowSize = 59;Esti...
Transformation (Gather Streams - Paralleli		3.30754	308	20.2056	62.0	AvgRowSize = 59;Esti...
Filter		3.30754	308	20.177	60.0	AvgRowSize = 59;Esti...
Value (Compute Scalar)		24806.1		20.1741		AvgRowSize = 59;Esti...
Value (Compute Scalar - Sequ		24806.1	28105	20.1735	60.0	AvgRowSize = 55;Esti...
Transformation (Segment)		24806.1	28105	20.173	59.0	AvgRowSize = 55;Esti...
Sort		24806.1	28105	20.1728	59.0	AvgRowSize = 47;Esti...
Transformation (Reparti		24806.1	28105	19.7544	56.0	AvgRowSize = 47;Esti...

Operations Tree



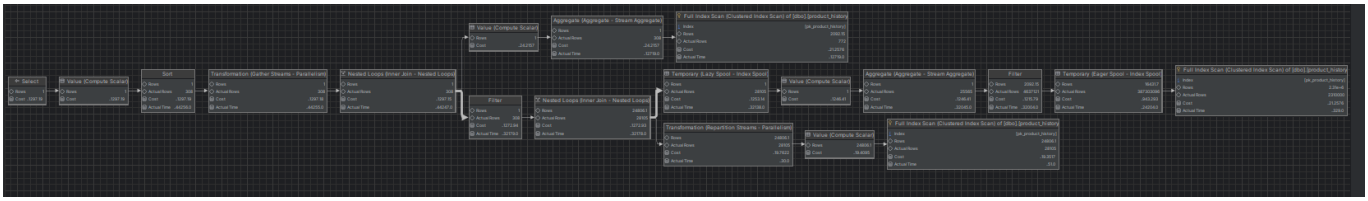
Session | north.dbo.product_history | Result 4

Output | Plan

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
Select		1		1297.19		CardinalityEstimation...
Value (Compute Scalar)		1		1297.19		AvgRowSize = 59;Es...
Sort		1	308	1297.19	44256.0	AvgRowSize = 55;Es...
Transformation (Gather Streams - Parallelism)		1	308	1297.18	44255.0	AvgRowSize = 55;Es...
Nested Loops (Inner Join - Nested Loop		1	308	1297.15	44247.0	AvgRowSize = 55;Es...
Filter		1	308	1272.94	32179.0	AvgRowSize = 51;Es...
Nested Loops (Inner Join - Nestec		24806.1	28105	1272.93	32178.0	AvgRowSize = 55;Es...
Transformation (Repartition Strean		24806.1	28105	19.7622	30.0	AvgRowSize = 51;Es...
Value (Compute Scalar)		24806.1		19.4095		AvgRowSize = 51;Es...

Operations Tree

168:1 LF UTF-8 4 spaces



Metoda	Koszt	Czas (ms)
Funkcje okna	20.2169	62.0
Bez funkcji okna	1297.19	44256.0

Dla tak ograniczonych danych już możemy zauważyć ogromną różnicę w czasie i koszcie wykonania tych zapytań.

PostgreSql

Od początku dane były ograniczoe, mimo tego dla sposobu bez funkcji okna nie udało się uzyskać wyników.

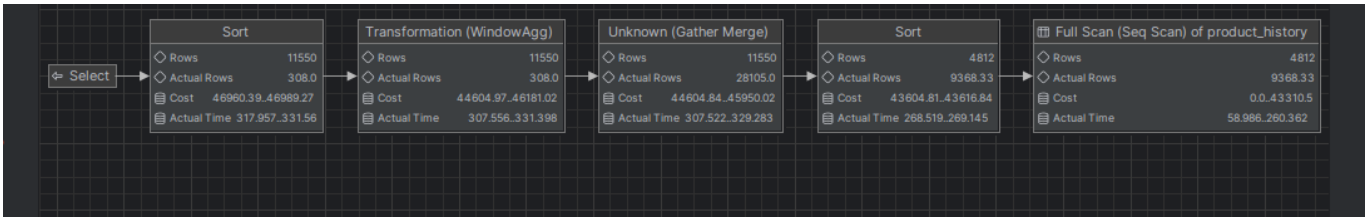
```
WITH rankedprices AS (
    SELECT
        EXTRACT(YEAR FROM date) AS year,
        productid,
        productname,
        unitprice,
        date,
```

```
        ROW_NUMBER() OVER (
            PARTITION BY productid, EXTRACT(YEAR FROM date)
            ORDER BY unitprice DESC
            ) AS ranknumber
    FROM product_history
    WHERE EXTRACT(YEAR FROM date) = 1997
)
SELECT *
FROM rankedprices
WHERE ranknumber <= 4
ORDER BY year, productid, ranknumber;
```

Session | Result 2

Output | Plan

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
Select								Planning = {...
Sort		11550	308.0	46989.27	331.56	46960.39	317.957	Parallel Awa...
Transformation (Wi		11550	308.0	46181.02	331.398	44604.97	307.556	Parent Relat...
Unknown (Gathe		11550	28105.0	45950.02	329.283	44604.84	307.522	Parent Relat...
Sort		4812	9368.33	43616.84	269.145	43604.81	268.519	Parent Relat...
Full Sca	table: product_histo...	4812	9368.33	43310.5	260.362	0.0	58.986	Parent Relat...




```
SELECT
    ph1.year,
    ph1.productid,
    ph1.productname,
    ph1.unitprice,
    ph1.date,
    (
        SELECT COUNT(*)
        FROM product_history ph2
        WHERE ph2.productid = ph1.productid
          AND EXTRACT(YEAR FROM ph2.date) = 1997
          AND ph2.unitprice >= ph1.unitprice
    ) AS ranknumber
FROM (
         SELECT
             productid,
             EXTRACT(YEAR FROM date) AS year,
             unitprice,
             productname,
             date
         FROM product_history
```

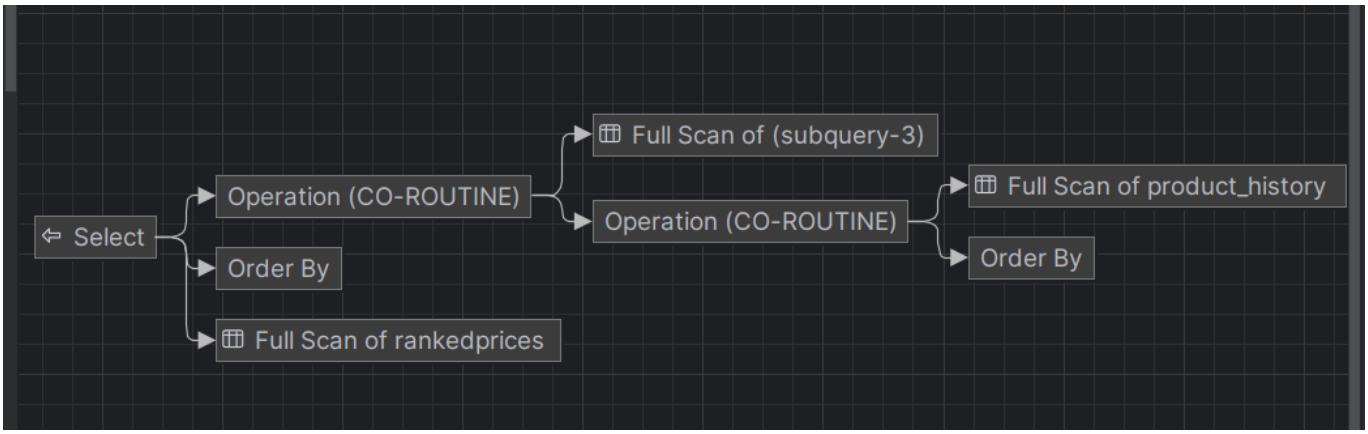
```
            WHERE EXTRACT(YEAR FROM date) = 1997
        ) ph1
WHERE (
            SELECT COUNT(*)
            FROM product_history ph2
            WHERE ph2.productid = ph1.productid
              AND EXTRACT(YEAR FROM ph2.date) = 1997
              AND ph2.unitprice > ph1.unitprice
        ) < 4
ORDER BY ph1.year, ph1.productid, ranknumber;
```

Metoda	Koszt	Czas (ms)
Funkcje okna	46989.27	331.56
Bez funkcji okna	przerwane	przerwane

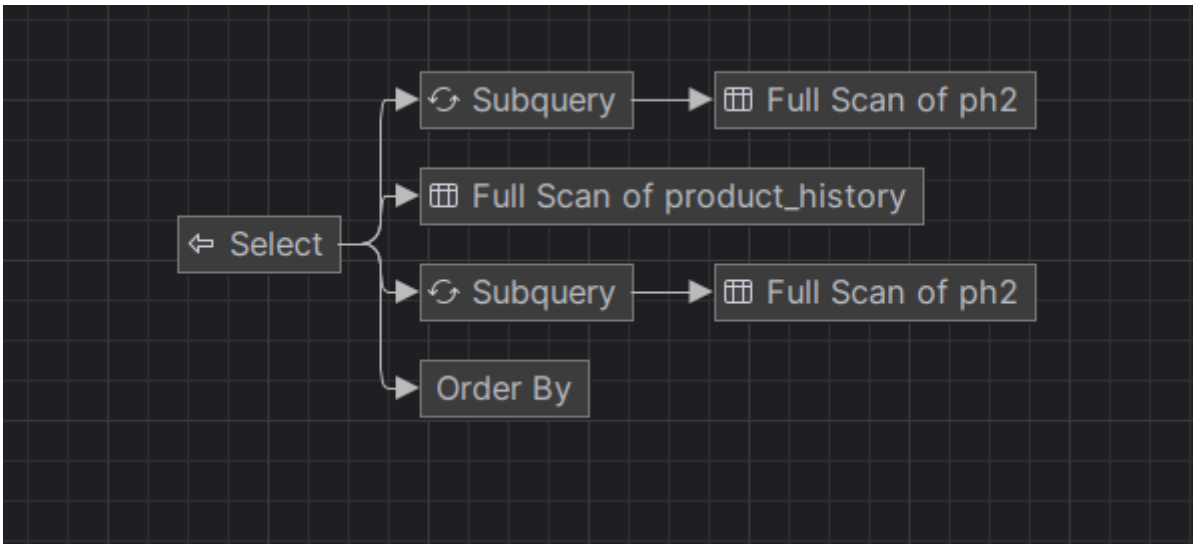
SQLite

Również działamy od razu na ograniczonych danych.

```
WITH rankedprices AS (
    SELECT
        strftime('%Y', date) AS year,
        productid,
        productname,
        unitprice,
        date,
        ROW_NUMBER() OVER (
            PARTITION BY productid, strftime('%Y', date)
            ORDER BY unitprice DESC
            ) AS ranknumber
    FROM product_history
    WHERE strftime('%Y', date) = '1997'
)
SELECT *
FROM rankedprices
WHERE ranknumber <= 4
ORDER BY year, productid, ranknumber;
```




```sql
SELECT
    ph1.year,
    ph1.productid,
    ph1.productname,
    ph1.unitprice,
    ph1.date,
    (
        SELECT COUNT(*)
        FROM product_history ph2
        WHERE ph2.productid = ph1.productid
          AND strftime('%Y', ph2.date) = '1997'
          AND ph2.unitprice >= ph1.unitprice
    ) AS ranknumber
FROM (
         SELECT
             productid,
             strftime('%Y', date) AS year,
             unitprice,
             productname,
             date
         FROM product_history
         WHERE strftime('%Y', date) = '1997'
     ) ph1
WHERE (
          SELECT COUNT(*)
          FROM product_history ph2
          WHERE ph2.productid = ph1.productid
            AND strftime('%Y', ph2.date) = '1997'
            AND ph2.unitprice > ph1.unitprice
      ) < 4
ORDER BY ph1.year, ph1.productid, ranknumber;
```



Metoda	Czas (ms)
Funkcje okna	1.868
Bez funkcji okna	przerwane

Zadanie 3

Funkcje `lag()`, `lead()`

Wykonaj polecenia, zaobserwuj wynik. Jak działają funkcje `lag()`, `lead()`

```
select productid, productname, categoryid, date, unitprice,
       lag(unitprice) over (partition by productid order by date)
as previousprodprice,
       lead(unitprice) over (partition by productid order by date)
as nextprodprice
from product_history
where productid = 1 and year(date) = 2022
order by date;

with t as (select productid, productname, categoryid, date, unitprice,
                  lag(unitprice) over (partition by productid
order by date) as previousprodprice,
                  lead(unitprice) over (partition by productid
order by date) as nextprodprice
           from product_history
           )
select * from t
where productid = 1 and year(date) = 2022
order by date;
```

Jak działają funkcje `lag()`, `lead()`?

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

> Wyniki:

MS SQL Server

Funkcja `lag()` (opóźnienie) pobiera wartość z poprzedniego wiersza w określonym oknie. Funkcja `lead()` (wyprzedzenie) pobiera wartość z następnego wiersza w określonym oknie. Domyślnie obie funkcje przeskakują o 1 wiersz, ale można ten parametr zmienić. Jeśli poprzedniego lub następnego wiersza nie ma to funkcja zwróci NULL.

```
select productid, productname, categoryid, date, unitprice,
       lag(unitprice) over (partition by productid order by date)
as previousprodprice,
       lead(unitprice) over (partition by productid order by date)
as nextprodprice
from product_history
where productid = 1 and year(date) = 2022
order by date;
```

	productid	productname	categoryid	date	unitprice	previousprodprice	nextprodprice
1	1	Chai	1	2022-01-01	16.34	<null>	11.18
2	1	Chai	1	2022-01-02	11.18	16.34	10.50
3	1	Chai	1	2022-01-03	10.50	11.18	20.48
4	1	Chai	1	2022-01-04	20.48	10.50	12.79
48	1	Chai	1	2022-02-17	18.87	10.11	21.69
49	1	Chai	1	2022-02-18	21.69	18.87	10.63
50	1	Chai	1	2022-02-19	10.63	21.69	<null>

Wartości NULL na skrajnych pozycjach wynikają z faktu, że baza najpierw odfiltrowuje dane zostawiając tylko 2022 rok, a dopiero na tak przycietym zbiorze uruchamia funkcje okna. Z tego powodu funkcja `lag()` w pierwszym wierszu nie widzi odciętych danych z 2021 roku i musi zwrócić wartość pustą. Z kolei w ostatnim wierszu funkcja `lead()` nie ma już dostępu do rekordów z 2023 roku, co analogicznie skutkuje wynikiem NULL.

```
with t as (select productid, productname, categoryid, date, unitprice,
                  lag(unitprice) over (partition by productid
order by date) as previousprodprice,
                  lead(unitprice) over (partition by productid
order by date) as nextprodprice
           from product_history
           )
select * from t
where productid = 1 and year(date) = 2022
order by date;
```

	productid	productname	categoryid	date	unitprice	previousprodprice	nextprodprice
1	1	Chai	1	2022-01-01	16.34	15.97	11.18
2	1	Chai	1	2022-01-02	11.18	16.34	10.50
3	1	Chai	1	2022-01-03	10.50	11.18	20.48
4	1	Chai	1	2022-01-04	20.48	10.50	12.79
47	1	Chai	1	2022-02-16	10.11	10.38	18.87
48	1	Chai	1	2022-02-17	18.87	10.11	21.69
49	1	Chai	1	2022-02-18	21.69	18.87	10.63
50	1	Chai	1	2022-02-19	10.63	21.69	<null>

W zapytaniu z klauzulą WITH funkcje okna obliczane są dla całego `product_history` jeszcze przed nałożeniem filtru na rok 2022. Dzięki temu funkcja `lag()` w pierwszym wierszu poprawnie zaciąga cenę z końca 2021 roku, która była dostępna przed filtrowaniem. Z kolei NULL dla funkcji `lead()` w ostatnim wierszu wynika wyłącznie z faktu, ze w samej tabeli fizycznie brakuje danych z 2023 roku, z których można by pobrać kolejną wartość.

Zapytanie równoważne bez użycia funkcji okna.

```sql
select
    p1.productid,
    p1.productname,
    p1.categoryid,
    p1.date,
    p1.unitprice,

    (select top 1 p2.unitprice
     from product_history p2
     where p2.productid = p1.productid
       and p2.date < p1.date
     order by p2.date desc
     ) as previousprodprice,

    (select top 1 p3.unitprice
     from product_history p3
     where p3.productid = p1.productid
       and p3.date > p1.date
     order by p3.date asc
     ) as nextprodprice

from product_history p1
where p1.productid = 1
  and YEAR(p1.date) = 2022
order by p1.date;
```

	productid	productname	categoryid	date	unitprice	previousprodprice	nextprodprice
1	1	Chai	1	2022-01-01	16.34	15.97	11.18
2	1	Chai	1	2022-01-02	11.18	16.34	10.50
3	1	Chai	1	2022-01-03	10.50	11.18	20.48
4	1	Chai	1	2022-01-04	20.48	10.50	12.79
47	1	Chai	1	2022-02-16	10.11	10.38	18.87
48	1	Chai	1	2022-02-17	18.87	10.11	21.69
49	1	Chai	1	2022-02-18	21.69	18.87	10.63
50	1	Chai	1	2022-02-19	10.63	21.69	<null>

Wynik tego zapytania jest dokładnie taki sam jak w przypadku zapytania z klauzulą WITH i funkcjami okna.

Metoda	Koszt	Czas (ms)
Funkcje okna (bezpośrednio z WHERE)	19.3605	18.0
Funkcje okna (w klauzuli WITH)	20.1186	105.0
Bez funkcji okna	204.898	1151.0

Wydajność zależy od momentu filtrowania danych. Bezpośrednie użycie funkcji okna jest najszybsze, bo silnik działa na już przyciętym zbiorze, podczas gdy klauzula WITH zmusza go do wcześniejszego przetworzenia całej historii. Z kolei podzapytania bez funkcji okna są skrajnie nieefektywne, ponieważ wymuszaja wielokrotne skanowanie tabeli wiersz po wierszu

---

Zadanie 4

Baza: Northwind, tabele customers, orders, order details

Napisz polecenie które wyświetla inf. o zamówieniach

Zbiór wynikowy powinien zawierać:

- nazwę klienta, nr zamówienia,
- datę zamówienia,
- wartość zamówienia (wraz z opłatą za przesyłkę),
- nr poprzedniego zamówienia danego klienta,
- datę poprzedniego zamówienia danego klienta,
- wartość poprzedniego zamówienia danego klienta.

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

> Wyniki:

```
;WITH ordervalues as (
    SELECT
        o.orderid,
        o.customerid,
        o.orderdate,
        SUM(od.unitprice * od.quantity * (1 - od.discount)) + o.freight AS
ordertotal
    FROM orders o
    JOIN orderdetails od ON o.orderid = od.orderid
    GROUP BY o.orderid, o.customerid, o.orderdate, o.freight
)
SELECT
    c.companyname as clientname,
    ov.orderid,
    ov.orderdate,
    ov.ordertotal,
    LAG(ov.orderid) OVER (PARTITION BY ov.customerid ORDER BY ov.orderdate)
```

```
AS previousorderid,
    LAG(ov.orderdate) OVER (PARTITION BY ov.customerid ORDER BY
ov.orderdate) AS previousorderdate,
    LAG(ov.ordertotal) OVER (PARTITION BY ov.customerid ORDER BY
ov.orderdate) AS previousordertotal
FROM ordervalues ov
JOIN customers c ON ov.customerid = c.customerid
ORDER BY c.companyname, ov.orderdate;
```

Zadanie 5

Funkcje `first_value()`, `last_value()`

Baza: Northwind, tabele customers, orders, order details

Wykonaj polecenia, zaobserwuj wynik. Jak działają funkcje `first_value()`, `last_value()`.

Skomentuj uzyskane wyniki. Czy funkcja `first_value` pokazuje w tym przypadku najdroższy produkt w danej kategorii, czy funkcja `last_value()` pokazuje najtańszy produkt?

Co jest przyczyną takiego działania funkcji `last_value`.

Co trzeba zmienić żeby funkcja last_value pokazywała najtańszy produkt w danej kategorii?

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

```
select productid, productname, unitprice, categoryid,
    first_value(productname) over (partition by categoryid
order by unitprice desc) first,
    last_value(productname) over (partition by categoryid
order by unitprice desc) last
from products
order by categoryid, unitprice desc;
```

> Wyniki:

```
select productid, productname, unitprice, categoryid,
    first_value(productname) over (partition by categoryid
order by unitprice desc) first,
    last_value(productname) over (partition by categoryid
order by unitprice desc) last
from products
order by categoryid, unitprice desc;
```

	productid	productname	unitprice	categoryid	first	last
1	38	Côte de Blaye	263.5	1	Côte de Blaye	Côte de Blaye
2	43	Ipoh Coffee	46	1	Côte de Blaye	Ipoh Coffee
3	2	Chang	19	1	Côte de Blaye	Chang
4	1	Chai	18	1	Côte de Blaye	Chartreuse verte
5	76	Lakkalikööri	18	1	Côte de Blaye	Chartreuse verte
6	35	Steeleye Stout	18	1	Côte de Blaye	Chartreuse verte
7	39	Chartreuse verte	18	1	Côte de Blaye	Chartreuse verte
8	70	Outback Lager	15	1	Côte de Blaye	Outback Lager
9	34	Sasquatch Ale	14	1	Côte de Blaye	Laughing Lumberjack Lager
10	67	Laughing Lumberjack Lager	14	1	Côte de Blaye	Laughing Lumberjack Lager
11	75	Rhönbräu Klosterbier	7.75	1	Côte de Blaye	Rhönbräu Klosterbier

Funkcja `first_value` zwraca nazwę najdroższego produktu w każdym wierszu danej kategorii. Funkcja `last_value` zwraca nazwę produktu z bieżącego wiersza, natomiast w przypadku remisów, czyli w tym przypadku produktów o tych samych cenach, zwraca nazwę ostatniego produktu w tej cenie. Nie pokazuje najtańszego produktu.

Przyczyną takiego działania funkcji `last_value` jest domyślna ramka okna, która przy użyciu klauzuli `ORDER BY` ustawia się automatycznie na `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, co oznacza, że obejmuje wszystkie wiersze od początku partycji aż do bieżącego wiersza włącznie z jego remisami. W rezultacie funkcja nie widzi całej kategorii od razu, lecz jej wzrok urywa się na aktualnie przetwarzanym wierszu. Dlatego dla każdego kroku jako ostatnią wartość bierze po prostu bieżący rekord lub ostatni z grupy remisującej.

Aby funkcja `last_value` faktycznie pokazywała najtańszy produkt na samym dole kategorii, musimy ręcznie nadpisać tę domyślną, uciętą ramkę okna i kazać bazie patrzeć do samego końca grupy. W tym celu po klauzuli ORDER BY należy dopisać: `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` czyli ramka okna od pierwszego wiersza aż do ostatniego wiersza partycji.

```sql
select productid, productname, unitprice, categoryid,
    first_value(productname) over (
        partition by categoryid
        order by unitprice desc
    ) as first_product,

    last_value(productname) over (
        partition by categoryid
        order by unitprice desc
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) as last_product
from products
order by categoryid, unitprice desc;
```

	productid	productname	unitprice	categoryid	first_product	last_product
1	38	Côte de Blaye	263.5	1	Côte de Blaye	Guaraná Fantástica
2	43	Ipoh Coffee	46	1	Côte de Blaye	Guaraná Fantástica
3	2	Chang	19	1	Côte de Blaye	Guaraná Fantástica
4	1	Chai	18	1	Côte de Blaye	Guaraná Fantástica
5	76	Lakkalikööri	18	1	Côte de Blaye	Guaraná Fantástica
6	35	Steeleye Stout	18	1	Côte de Blaye	Guaraná Fantástica
7	39	Chartreuse verte	18	1	Côte de Blaye	Guaraná Fantástica
8	70	Outback Lager	15	1	Côte de Blaye	Guaraná Fantástica
9	34	Sasquatch Ale	14	1	Côte de Blaye	Guaraná Fantástica
10	67	Laughing Lumberjack Lager	14	1	Côte de Blaye	Guaraná Fantástica
11	75	Rhönbräu Klosterbier	7.75	1	Côte de Blaye	Guaraná Fantástica
12	24	Guaraná Fantástica	4.5	1	Côte de Blaye	Guaraná Fantástica
13	63	Vegie-spread	43.9	2	Vegie-spread	Aniseed Syrup

Teraz funkcja `first_value` pokazuje najdroższy produkt w kategorii, a funkcja `last_value` najtańszy.

Zadanie 6

Baza: Northwind, tabele orders, order details

Napisz polecenie które wyświetla inf. o zamówieniach

Zbiór wynikowy powinien zawierać:

- Id klienta,
- nr zamówienia,
- datę zamówienia,
- wartość zamówienia (wraz z opłatą za przesyłkę),
- dane zamówienia klienta o najniższej wartości w danym miesiącu
    - nr zamówienia o najniższej wartości w danym miesiącu
    - datę tego zamówienia
    - wartość tego zamówienia
- dane zamówienia klienta o najwyższej wartości w danym miesiącu
    - nr zamówienia o najniższej wartości w danym miesiącu
    - datę tego zamówienia
    - wartość tego zamówienia

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

> Wyniki:

```
;WITH ordervalues as (
    SELECT
        o.orderid,
        o.customerid,
        o.orderdate,
        SUM(od.unitprice * od.quantity * (1 - od.discount)) + o.freight AS
ordertotal,
        YEAR(o.orderdate) AS orderyear,
        MONTH(o.orderdate) AS ordermonth
    FROM orders o
    JOIN orderdetails od ON o.orderid = od.orderid
    GROUP BY o.orderid, o.customerid, o.orderdate, o.freight
)
SELECT
    ov.customerid as clientid,
    ov.orderid,
    ov.orderdate,
    ov.ordertotal,
    FIRST_VALUE(ov.orderid) OVER (
        PARTITION BY ov.customerid, ov.orderyear, ov.ordermonth
        ORDER BY ov.ordertotal ASC
```

```
        ) AS minorderid,
    FIRST_VALUE(ov.orderdate) OVER (
        PARTITION BY ov.customerid, ov.orderyear, ov.ordermonth
        ORDER BY ov.ordertotal ASC
        ) AS minorderdate,
    FIRST_VALUE(ov.ordertotal) OVER (
        PARTITION BY ov.customerid, ov.orderyear, ov.ordermonth
        ORDER BY ov.ordertotal ASC
        ) AS minordertotal,
    FIRST_VALUE(ov.orderid) OVER (
        PARTITION BY ov.customerid, ov.orderyear, ov.ordermonth
        ORDER BY ov.ordertotal DESC
        ) AS maxorderid,
    FIRST_VALUE(ov.orderdate) OVER (
        PARTITION BY ov.customerid, ov.orderyear, ov.ordermonth
        ORDER BY ov.ordertotal DESC
        ) AS maxorderdate,
    FIRST_VALUE(ov.ordertotal) OVER (
        PARTITION BY ov.customerid, ov.orderyear, ov.ordermonth
        ORDER BY ov.ordertotal DESC
        ) AS maxordertotal
FROM ordervalues ov
ORDER BY ov.customerid, ov.orderyear, ov.ordermonth, ov.orderid;
```

Zadanie 7

Baza: Northwind, tabela product_history

Napisz polecenie które pokaże wartość sprzedaży każdego produktu narastająco od początku każdego miesiąca. Użyj funkcji okna

Zbiór wynikowy powinien zawierać:

- id pozycji
- id produktu
- datę
- wartość sprzedaży produktu w danym dniu
- wartość sprzedaży produktu narastające od początku miesiąca

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna, porównaj wyniki, czasy i plany zapytań (koszty).

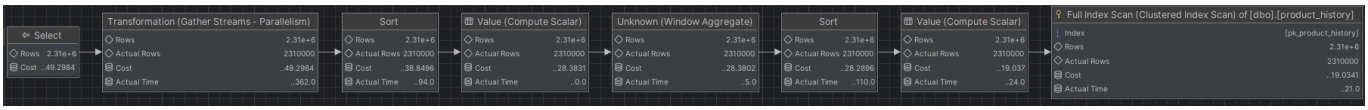
Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSql, SQLite)

> Wyniki:

MS SQL Server

Funkcja okna

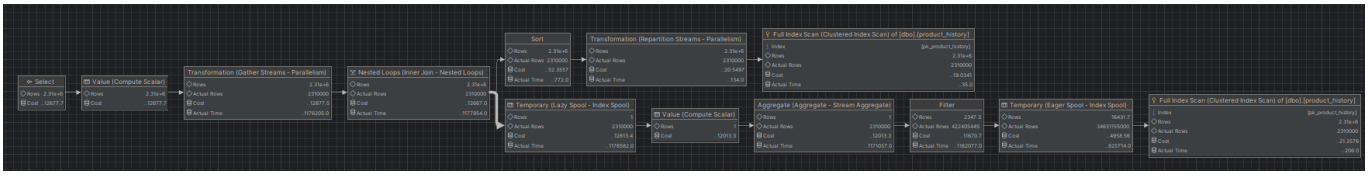
```sql
select
    id,
    productid,
    date,
    unitprice,
    sum(unitprice) over (
        partition by productid, year(date), month(date)
        order by date
    ) as mtd_sales
from product_history
order by productid, date
```



Plan jest liniowy i zoptymalizowany. Wykonuje zaledwie jeden skan tabeli, a sumę narastającą oblicza w jednym przejściu za pomocą dedykowanego operatora Window Aggregate.

Bez użycia funkcji okna

```sql
select
    p1.id,
    p1.productid,
    p1.date,
    p1.unitprice,
    (select sum(p2.unitprice)
     from product_history p2
     where p2.productid = p1.productid
       and year(p2.date) = year(p1.date)
       and month(p2.date) = month(p2.date)
       and p2.date <= p1.date
     ) as mtd_sales
from product_history p1
order by productid, date
```



Plan skrajnie nieefektywny i złożony. Głównym wąskim gardłem jest tu operator Nested Loops, który wymusza na bazie wielokrotne, bardzo kosztowne skanowanie tej samej tabeli dla każdego wiersza z osobna, na planie również dwa Full Index Scany.

Metoda	Koszt	Czas (ms)
Funkcja okna	49.2984	374

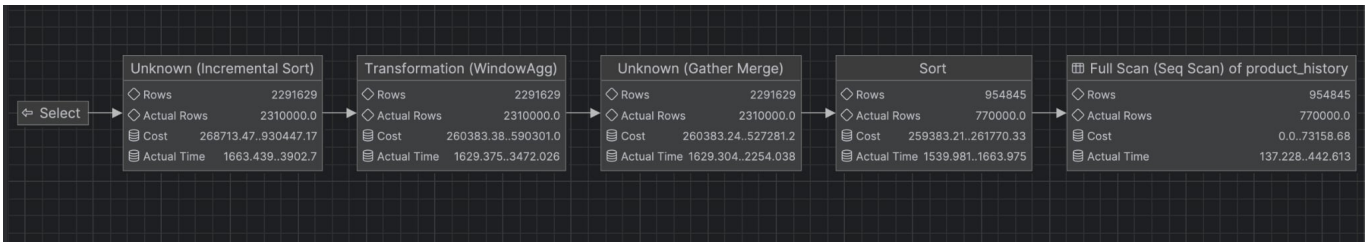
Metoda	Koszt	Czas (ms)
Bez funkcji okna	12677.7	1176205

Zapytanie z funkcją okna poradziło sobie znacznie lepiej niż to bez funkcji okna. Zarówno czas jak i koszt są drastycznie mniejsze dla funkcji okna.

PostgreSql

Funkcja okna

```
select
    id,
    productid,
    date,
    unitprice,
    sum(unitprice) over (
        partition by productid, extract(year from date), extract(month from
date)
        order by date
    ) as mtd_sales
from product_history
order by productid, date;
```

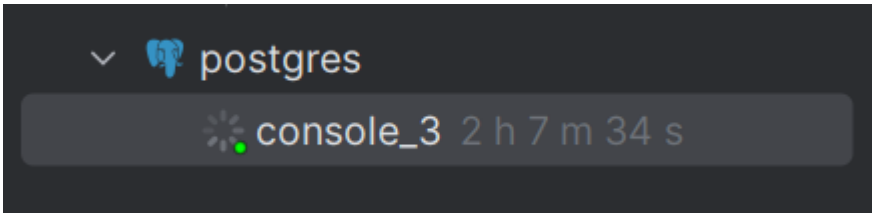


Plan jest zoptymalizowany i liniowy, oparty na jednokrotnym skanowaniu tabeli Seq Scan i zrównoleglonym sortowaniu Gather Merge. Dzięki zastosowaniu dedykowanego operatora WindowAgg, suma narastająca obliczana jest wydajnie w jednym przejściu przez posortowane dane.

Bez użycia funkcji okna - podzapytanie

```
select
    p1.id,
    p1.productid,
    p1.date,
    p1.unitprice,

    (select sum(p2.unitprice)
     from product_history p2
     where p2.productid = p1.productid
       and extract(year from p2.date) = extract(year from p1.date)
       and extract(month from p2.date) = extract(month from p1.date)
       and p2.date <= p1.date
```

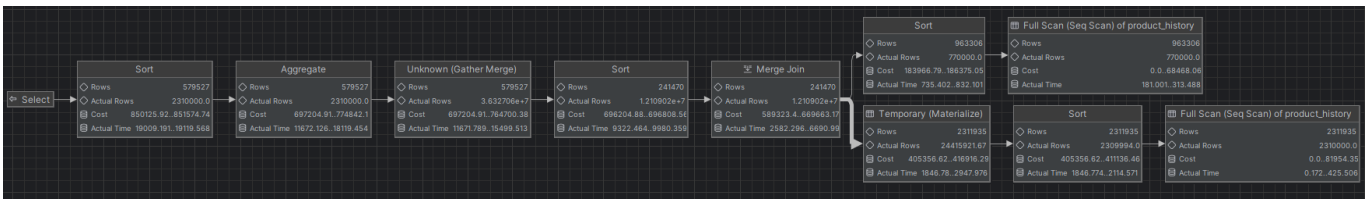
```
        ) as mtd_sales

  from product_history p1
  order by productid, date;
```



Zapytanie okazało się skrajnie niewydajne - jego wykonanie zostało przerwane po 2 godzinach. Główną przyczyną tak drastycznego spadku wydajności jest zastosowanie podzapytania skorelowanego, które zmusza optymalizator do wielokrotnego, pełnego skanowania tabeli dla każdego analizowanego wiersza z osobna. Dodatkowy, choć mniejszy narzut obliczeniowy, generuje wielokrotne wywoływanie funkcji extract w warunkach złączenia.

Kolejna próba bez użycia funkcji okna - join

```
  select
      p1.id,
      p1.productid,
      p1.date,
      p1.unitprice,
      sum(p2.unitprice) as mtd_sales
  from product_history p1
  join product_history p2
      on p1.productid = p2.productid
      and extract(year from p1.date) = extract(year from p2.date)
      and extract(month from p1.date) = extract(month from p2.date)
      and p2.date <= p1.date
  group by
      p1.id,
      p1.productid,
      p1.date,
      p1.unitprice
  order by
      p1.productid,
      p1.date;
```



Plan opiera się na złączeniu dwóch pełnych skanów tabeli za pomocą operatora Merge Join. Choć rozwiązanie to eliminuje problem zapętlonego podzapytania, warunek nierówności (<=) generuje

gigantyczną liczbę wierszy pośrednich. Wymusza to na bazie kosztowne operacje wielokrotnego sortowania i grupowania, co czyni tę metodę wciąż znacznie wolniejszą od zastosowania funkcji okna.

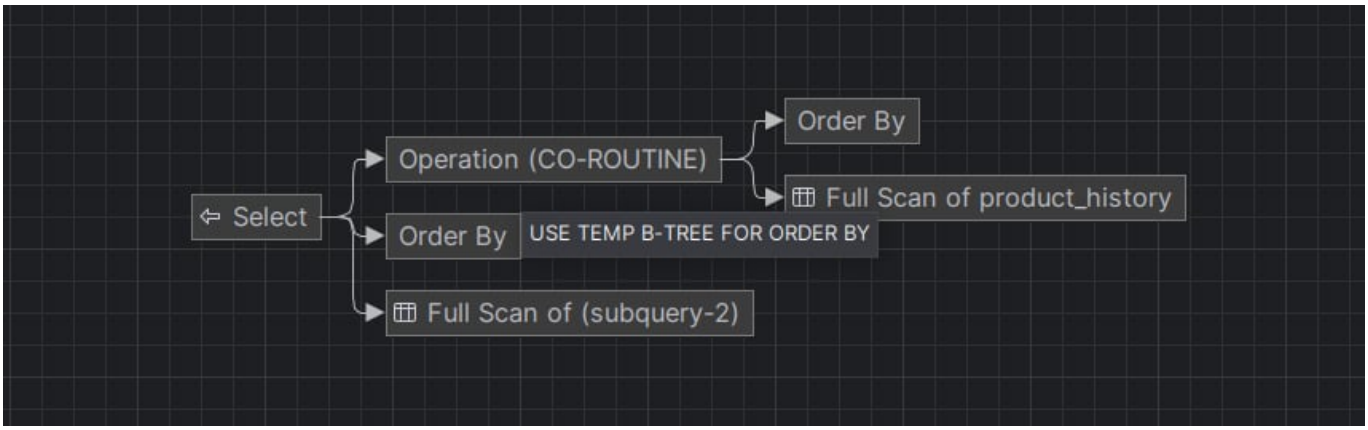
Metoda	Koszt	Czas
Funkcja okna	938197.96	2364.96 ms
Bez funkcji okna (join)	851574.74	19119.568 ms
Bez funkcji okna (podzapytanie)	-	Przerwane po 2h

Funkcja okna wypada najlepiej po względem czasu wykonania, podczas gdy podzapytanie skorelowane okazało się zupełną katastrofą. Warto jednak zauważyć, że metoda ze złączeniem poradziła sobie wcale nie tak źle (ok 19 s), stanowiąc całkiem sensowną, choć zauważalnie wolniejszą alternatywę.

SQLite

Funkcja okna

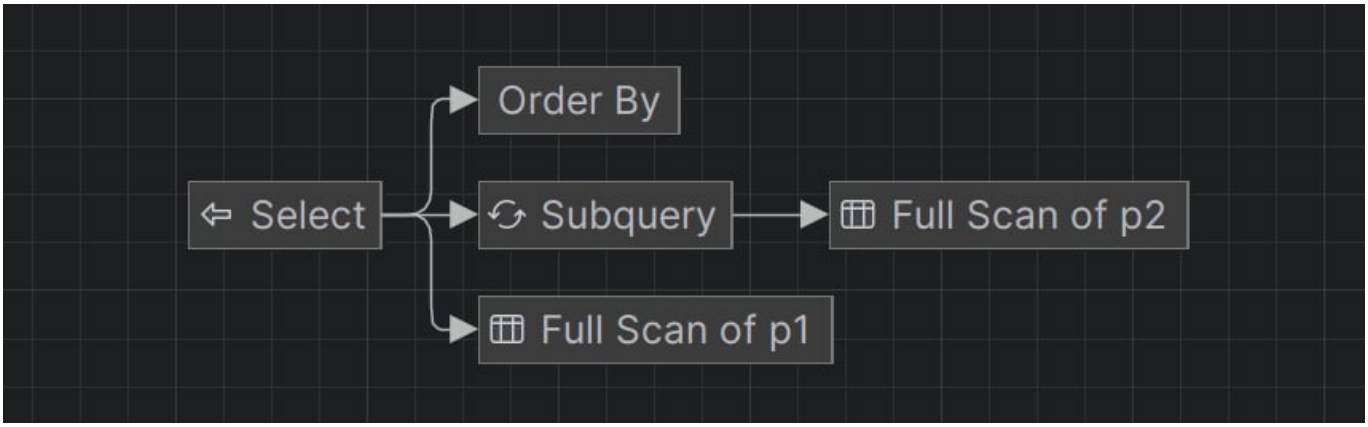
```
select
    id,
    productid,
    date,
    unitprice,
    sum(unitprice) over (
        partition by productid, strftime('%Y-%m', date)
        order by date
    ) as mtd_sales
from product_history
order by productid, date;
```



Plan wysoce zoptymalizowany. Wykonuje tylko jeden skan tabeli, a obliczenia sumy narastającej realizuje wydajnie w pamięci przy użyciu operacji CO-ROUTINE oraz tymczasowego drzewa B-TREE do sortowania.

Bez użycia funkcji okna - podzapytanie

```
select
    p1.id,
    p1.productid,
    p1.date,
    p1.unitprice,

    (select sum(p2.unitprice)
     from product_history p2
     where p2.productid = p1.productid
       and strftime('%Y-%m', p2.date) = strftime('%Y-%m', p1.date)
       and p2.date <= p1.date
    ) as mtd_sales

from product_history p1
order by p1.productid, p1.date;
```

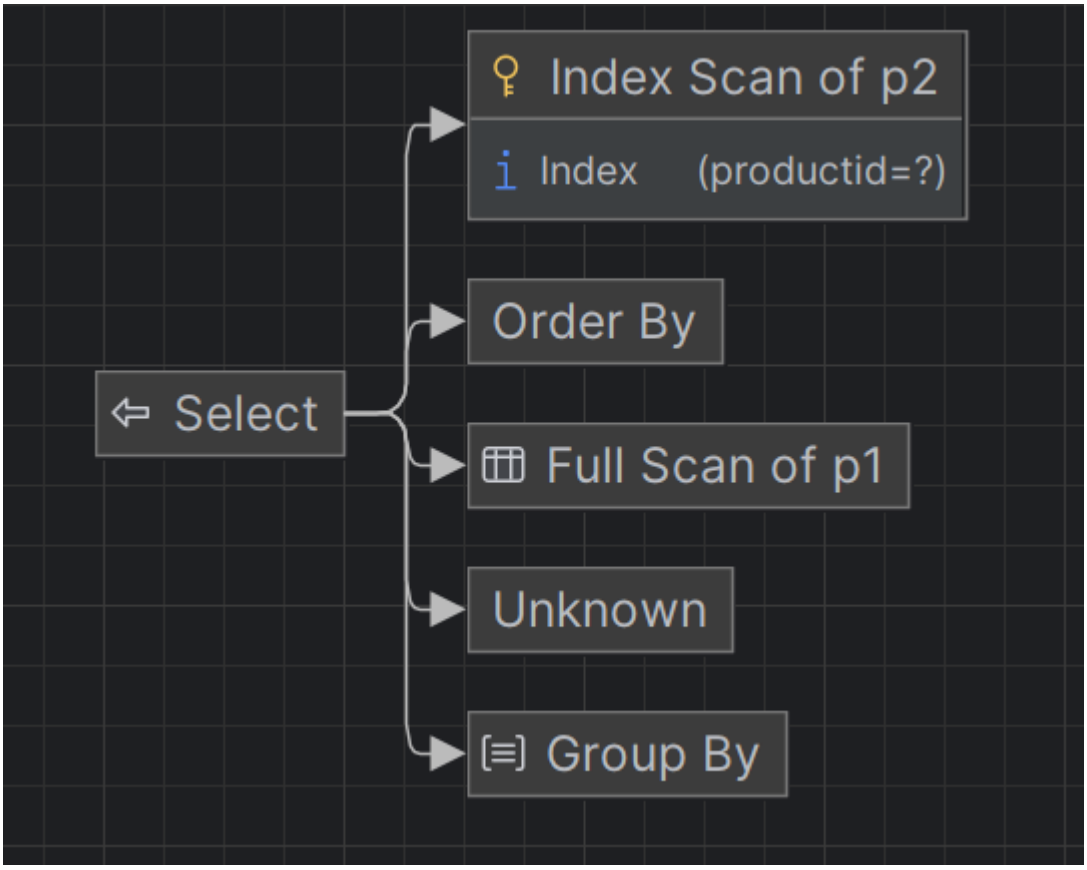


Plan skrajnie niewydajny. Baza wykonuje pełny skan tabeli głównej, a dla każdego napotkanego wiersza uruchamia podzapytanie wymuszające kolejne, powtarzające się pełne skanowanie tabeli p2.

Bez użycia funkcji okna - join

```
select
    p1.id,
    p1.productid,
    p1.date,
    p1.unitprice,
    sum(p2.unitprice) as mtd_sales
from product_history p1
join product_history p2
    on p1.productid = p2.productid
    and strftime('%Y-%m', p1.date) = strftime('%Y-%m', p2.date)
    and p2.date <= p1.date
group by
    p1.id,
    p1.productid,
    p1.date,
    p1.unitprice
order by
```

```
    p1.productid,
    p1.date;
```



Plan wykorzystuje pełny skan tabeli p1 i skan indeksu dla p2. Niestety, złączenie z warunkiem nierówności generuje gigantyczną liczbę wierszy pośrednich, co wymusza niezwykle kosztowne operacje grupowania i sortowania, ostatecznie dławiąc silnik SQLite równie mocno co podzapytanie.

Metoda	Czas
Funkcja okna	3687 ms
Bez funkcji okna (podzapytanie)	Przerwane po 40min
Bez funkcji okna (join)	Przerwane po 30min

Kolejny raz funkcje okna wypadają najlepiej, można powiedzieć, że w tym przypadku są bezkonkurencyjne.

Podsumowanie czasów dla każdego z serwerów

Metoda	MS SQL Server	PostgreSQL	SQLite
Funkcja okna	374 ms	2364.96 ms	3687 ms
Bez funkcji okna (podzapytanie)	1176205 ms	Przerwane	Przerwane

Funkcje okna są nieporównywalnie szybsze w każdym z testowanych serweró. Podejście z podzapytaniem kompletnie nie radzi sobie przy większej ilości danych, MS SQL Server mielił to zapytanie przez prawie 20

minut, a w Postgresie i SQLite trzeba było je po prostu przerwać.

Zadanie 8

Wykonaj kilka "własnych" przykładowych analiz.

Czy są jeszcze jakieś ciekawe/przydatne funkcje okna (z których nie korzystałeś w ćwiczeniu)? Spróbuj ich użyć w zaprezentowanych przykładach.

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

> Wyniki:

Średnia wartości zamówień z ostatnich 5 zamówień klienta.

```
SELECT
    o.customerid,
    o.orderid,
    o.orderdate,
    SUM(od.unitprice * od.quantity * (1 - od.discount)) + o.freight AS
ordertotal,
    AVG(SUM(od.unitprice * od.quantity * (1 - od.discount)) + o.freight)
        OVER (
            PARTITION BY o.customerid
            ORDER BY o.orderdate
            ROWS BETWEEN 4 PRECEDING AND CURRENT ROW
            ) AS moving_avg
FROM orders o
JOIN orderdetails od ON o.orderid = od.orderid
GROUP BY o.customerid, o.orderid, o.orderdate, o.freight
ORDER BY o.customerid, o.orderdate;
```

CUME_DIST() - procent zamówień mniejszych lub równych danej wartości

Dla każdego zamówienia klienta liczymy pozycję zamówienia w procentach w stosunku do wszystkich zamówień tego klienta.

```
SELECT
    o.customerid,
    o.orderid,
    o.orderdate,
    SUM(od.unitprice * od.quantity * (1 - od.discount)) + o.freight AS
ordertotal,
    CUME_DIST() OVER (
        PARTITION BY o.customerid
        ORDER BY SUM(od.unitprice * od.quantity * (1 - od.discount)) +
o.freight
```

```
        ) AS percent_rank
FROM orders o
JOIN orderdetails od ON o.orderid = od.orderid
GROUP BY o.customerid, o.orderid, o.orderdate, o.freight
ORDER BY o.customerid, ordertotal, o.orderdate;
```

PERCENT_RANK() - ranking procentowy

Ranking zamówień klienta w stosunku do najwyższej wartości zamówienia.

```
SELECT
    o.customerid,
    o.orderid,
    SUM(od.unitprice * od.quantity * (1 - od.discount)) + o.freight AS
ordertotal,
    PERCENT_RANK() OVER (
        PARTITION BY o.customerid
        ORDER BY SUM(od.unitprice * od.quantity * (1 - od.discount)) +
o.freight
        ) AS percent_rank
FROM orders o
JOIN orderdetails od ON o.orderid = od.orderid
GROUP BY o.customerid, o.orderid, o.freight
ORDER BY o.customerid, ordertotal;
```

Umożliwia to szybkie wyświetlenie np. top 10% największych zamówień klienta:

```
WHERE percent_rank >= 0.9
```

Punktacja

zadanie	pkt
1	1
2	2
3	1
4	1
5	1
6	1
7	2
8	2

razem	11